

# Multi-Label Toxic Comment Classification with Fine-Tuned BERT

Course Report — NLP Final Project

Dataset: Kaggle Jigsaw Toxic Comment Classification (Wikipedia talk-page comments)

Author: Sameer Batra

---

## 1. Introduction

### 1.1 Overview

Online discussion spaces routinely need automated assistance to surface **harassing, obscene, threatening, or identity-based hostile** language at scale. The Jigsaw Toxic Comment Classification task frames this as **multi-label text classification**: each comment is associated with a subset of six binary toxicity attributes — toxic, severe\_toxic, obscene, threat, insult, and identity\_hate — rather than a single mutually exclusive class.

This project builds an end-to-end pipeline that:

- Ingests and cleans raw Kaggle CSVs.
- Explores label imbalance, text length, and label co-occurrence via a structured EDA module.
- Trains a **BERT-base-uncased** encoder with a linear **multi-label head**, optimized with **class-aware focal loss** and standard regularization (dropout, weight decay).
- Evaluates using **ROC-AUC** (probability ranking) and **F1** (thresholded decisions at 0.5), with checkpointing and learning-rate scheduling driven by a configurable **monitor metric** (current default: **mean F1**).

## 2. Description of Individual Work (Algorithmic Background)

### 2.1 Problem Formulation

For a comment text  $x$  and labels  $y \in \{0, 1\}^K$  with  $K = 6$ , the model outputs logits  $z \in \mathbb{R}^K$ . Multi-label classification uses **sigmoid** activations per label (not softmax across labels):

$$P(y_k = 1 \mid x) = \sigma(z_k) = \frac{1}{1 + e^{-z_k}}$$

Training minimizes a sum (or mean) over labels of **binary** losses sharing the same encoder.

### 2.2 Encoder: BERT and the Classification Head

**BERT** (Devlin et al., 2019) maps tokenized input to contextual vectors. This project uses **bert-base-uncased**: 12 transformer layers, hidden size 768. The classifier takes the **final hidden state of the [CLS] token**:

$$h = \text{BERT}(x)_{[\text{CLS}]} \in \mathbb{R}^{768}$$

The implemented head (see *model.py*) applies **dropout** and a **linear** layer:

$$z = Wh + b, \quad W \in \mathbb{R}^{K \times 768}, \quad b \in \mathbb{R}^K$$

Optional **partial freezing** of the bottom  $n$  encoder layers is supported via `freeze_bert_layers` (not enabled in the current `train.py`).

### 2.3 Focal Loss for Imbalance

Jigsaw labels are **severely imbalanced** (e.g., many comments are non-toxic for all tags; rare tags such as *threat* have far fewer positives). **Focal loss** (Lin et al., 2017) modulates **binary cross-entropy** so that "easy" examples contribute less.

For one label, with logit  $z$ , target  $y \in \{0, 1\}$ , and  $p = \sigma(z)$ , **BCE with logits** underpins the focal construction. The **focal weight** is:

$$(1 - p_t)^\gamma, \quad p_t = py + (1 - p)(1 - y), \quad \gamma \geq 0$$

**Class-balanced**  $\alpha_t$  (per-label  $\alpha_k$  from training frequencies): for each label  $k$ ,

$$\alpha_k = \frac{N_{\text{neg}}}{N_{\text{total}}} = 1 - \text{positive rate}$$

The tensor is applied so that  $\alpha_t = \alpha_k$  when  $y = 1$  and  $1 - \alpha_k$  when  $y = 0$ . The implemented loss averages **across batch, labels, and items** with  $\gamma = 2.0$  (*focal\_loss.py*). **Intuition:** rare positives get larger  $\alpha$  on the positive class, and **hard examples** (where  $p_t$  is far from 1) receive a larger focal factor than easy ones.

## 2.4 Optimization and Scheduling

Training uses **AdamW** with learning rate  $2 \times 10^{-5}$  and **weight decay** 0.01. A **linear warmup** over the first 10% of optimizer steps is applied via Hugging Face *get\_constant\_schedule\_with\_warmup* (warmup then **constant** LR until plateau logic triggers decay). **Gradient clipping** (max norm 1) stabilizes transformer training.

**ReduceLROnPlateau** (mode='max') reduces the learning rate when the monitored validation metric stops improving; **early stopping** halts training after a configurable patience on the same metric.

## 2.5 Evaluation Metrics

**ROC-AUC** per label uses predicted probabilities vs. binary ground truth; it measures **ranking quality** and is insensitive to a fixed threshold.

**F1** per label uses **threshold 0.5** on  $\sigma(z)$ :

$$F_1 = \frac{2PR}{P+R}$$

with standard binary precision/recall on hard predictions. The codebase reports **macro-style means** *mean\_auc* and *mean\_f1* over labels that have **at least one positive** in the validation fold (so AUC/F1 are not computed on degenerate all-negative columns).

---

# 3. Detailed Description of Project Work

## 3.1 Pre-processing (data\_cleaner.py)

Comments mix **HTML**, **wiki markup** (links, templates), **URLs**, **IP addresses**, and **usernames**. The cleaner chains **single-responsibility** classes: HTML stripping (BeautifulSoup), wiki link/template/heading normalization, URL/IP/username anonymisation placeholders, Unicode NFKC normalization, whitespace normalization, and empty text  $\rightarrow$  [EMPTY].

## 3.2 Train / Validation / Test Protocol (dataset.py)

- **Train+val** come from *train\_clean.csv*; **test** is *test\_clean.csv* (inference-oriented split in code).
- **Stratified split** on the *toxic* column: **85% / 15%** train vs. validation (val\_size=0.15, random\_state=42).
- A representative log run: **train 135,635 · val 23,936 · test 153,164**.

Tokenization uses **BertTokenizerFast** with **max length 256**, **padding to max\_length**, and **truncation**.

## 3.3 Model (model.py, train.py)

- **Backbone:** BertModel.from\_pretrained("bert-base-uncased").

- **Head:** Dropout(0.3) → Linear(768 → 6).
- All encoder layers trained (freeze\_bert\_layers=0).

### 3.4 Training (trainer.py, train.py)

- **Batch size** 32; **epochs** 5.
- **Focal loss** with  $\gamma = 2$  and  $\alpha_k$  from the **post-split training dataframe only**.
- **Checkpointing** on configurable *monitor\_metric* (mean\_f1 or mean\_auc).
- **CSV log:** *training\_results.csv* (per-epoch losses and per-label AUC/F1).

## 4. Results

### 4.1 Training Dynamics — Example Run

Table 1 — Epoch-level validation summary (from training\_results.csv).

| Epoch | Train Loss | Val Loss | Mean ROC-AUC | Mean F1 |
|-------|------------|----------|--------------|---------|
| 1     | 0.0037     | 0.0023   | 0.9888       | 0.4223  |
| 2     | 0.0021     | 0.0022   | 0.9909       | 0.4382  |
| 3     | 0.0016     | 0.0033   | 0.9912       | 0.5416  |
| 4     | 0.0013     | 0.0033   | 0.9911       | 0.5358  |
| 5     | 0.0011     | 0.0035   | 0.9900       | 0.5129  |

### 4.2 Per-Label Metrics — Validation, Epoch 3 (Best Mean F1)

Table 2 — Per-label ROC-AUC and F1 (threshold  $\tau = 0.5$ ).

| Label         | ROC-AUC | F1    |
|---------------|---------|-------|
| toxic         | 0.9881  | 0.802 |
| severe_toxic  | 0.9910  | 0.381 |
| obscene       | 0.9943  | 0.766 |
| threat        | 0.9912  | 0.212 |
| insult        | 0.9905  | 0.731 |
| identity_hate | 0.9919  | 0.357 |

## 5. Summary and Conclusions

### 5.1 Summary

The project implements **structured cleaning**, **EDA**, a **BERT-base multi-label classifier** with **focal loss** and **class-wise**  $\alpha$ , and **ROC-AUC / F1** evaluation with **CSV logging** and **metric-driven checkpointing**. Example runs show **mean AUC**  $\approx 0.99$  on validation alongside **mean F1**  $\approx 0.54$  at the best epoch — illustrating the gap between **ranking** and **hard-threshold** metrics under

imbalance.

## 5.2 What We Learned

- **Imbalance** dominates practical F1 on rare labels even when AUC is high.
- **Choice of monitored metric** changes which weights are saved as "best."
- **Wiki-specific cleaning** stabilizes token statistics.

## 5.3 Future Improvements

1. Per-label thresholds or calibration on validation.
  2. Weighted sampling in addition to focal loss.
  3. Long-document handling if policy requires it.
  4. Robustness / adversarial spelling tests.
  5. Gold-label evaluation where available.
- 

## References

- Devlin, J., et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. NAACL, 2019.
- Lin, T.-Y., et al. *Focal Loss for Dense Object Detection*. ICCV, 2017.
- Kaggle Jigsaw Toxic Comment Classification Challenge (dataset).